Relationship between P, NP, BQP and BPP

Zhouyue Qian 3220104074

December 19, 2024

1 Definition of P, NP, BQP and BPP

• **P**:P represents the class of decision problems solvable by a deterministic Turing machine in polynomial time.
• **NP**:NP consists of decision problems for which a solution, if provided, can be verified in polynomial time by a deterministic Turing machine.
• **BQP**:BQP encompasses problems solvable by a quantum Turing machine in polynomial time, with an error probability of less than 1/3.
• **BPP**:BPP includes problems solvable by a probabilistic Turing machine in polynomial time with an error probability of less than 1/3 for all instances.
• ***PSPACE**:The class of decision problems that can be solved by a deterministic Turing machine using a polynomial amount of memory space.

2 Known relationship between P, NP, BQP and BPP

We have already known that $P \subseteq BPP \subseteq BQP$ and $P \subseteq NP \subseteq PSPACE$.

2.1 Proof of $P \subseteq BPP \subseteq BQP$

2.1.1 • Proof of $P \subseteq BPP$

For any language $L \in P$, construct a probabilistic Turing machine (PTM) M' such that:

- If $x \in L$, $\Pr[M'(x) \text{ accepts}] \geq 2/3$.
- If $x \notin L$, $\Pr[M'(x) \text{ rejects}] \geq 2/3$.

Additionally, M' must run in polynomial time. Let $L \in P$, and let M be a deterministic Turing machine that decides L in time $O(n^k)$ for some constant k. Define a PTM M' that, on input x, simulates M on x without using any randomness. Since M is deterministic and always gives the correct answer, M' will also always give the correct answer, regardless of any random bits it may have access to.

- If $x \in L$, $M'(x)$ will always accept, so $\Pr[M'(x) \text{ accepts}] = 1 \geq 2/3$.
- If $x \notin L$, $M'(x)$ will always reject, so $\Pr[M'(x) \text{ rejects}] = 1 \geq 2/3$.
- The running time of M' is the same as that of M, i.e., $O(n^k)$, which is polynomial.

Since M' meets all the criteria for L to be in BPP, we conclude that every language in P can be decided by a probabilistic Turing machine with bounded error in polynomial time. Therefore, $P \subseteq BPP$.

2.1.2 • Proof of $BPP \subseteq BQP$

• Simulation of Classical Probabilistic Algorithms by Quantum Computers

Quantum computers can efficiently simulate classical probabilistic computations. This is because quantum algorithms can mimic the randomness used in classical probabilistic algorithms by using measurements of qubits in certain states. The simulation of a classical probabilistic Turing machine by a quantum Turing machine can be done with only polynomial overhead, ensuring that the polynomial time complexity is preserved.

• Error Probability Preservation

The error probabilities inherent in the classical probabilistic algorithm are preserved in the quantum simulation. Since the quantum simulation replicates the probabilistic outcomes of the classical algorithm, the error bounds of at most 1/3 are maintained. Therefore, $BPP \subseteq BQP$.

2.2 Proof of $P \subseteq NP$

Let L be a language in P. By definition, there exists a deterministic Turing machine M that decides L in polynomial time, say $O(n^k)$ for some constant k.

To show $L \in NP$, we need to provide a verifier V that, given an input x and a certificate y, verifies whether $x \in L$ in polynomial time.

Define the verifier V as follows: V ignores the certificate y and simply simulates M on input x.

Since M runs in polynomial time, V also runs in polynomial time. Therefore, V correctly decides whether $x \in L$ in polynomial time without using the certificate. Therefore, $L \in NP$, and we conclude that $P \subseteq NP$.

2.3 Proof of $NP \subseteq PSPACE$

• Simulation of Non-deterministic Turing Machine:

Consider a non-deterministic Turing machine (NTM) that runs in time $O(n^k)$ for some constant k. The total number of configurations of this NTM is polynomial in n, specifically $O(n^k)$, since each step can only change the tape and head position in a limited way.

• Depth-First Search Simulation:

A deterministic Turing machine (DTM) can simulate the NTM by exploring the computation paths using depth-first search (DFS). DFS can be implemented in space proportional to the depth of the recursion, which is polynomial in n ($O(n^k)$).

• Space Complexity:

The space used to represent each configuration is polynomial in n. Therefore, the total space used by the DTM to simulate the NTM is polynomial in n.

• Acceptance Condition:

The NTM accepts if there exists at least one computation path leading to acceptance. The DTM can check for the existence of such a path using DFS within polynomial space.

Therefore, $NP \subseteq PSPACE$.

3 Relationship between NP and BQP

To figure out the relationship between P,NP,BPP and BQP, we need to find out the relationship between NP and BQP. Nevertheless, the relationship between NP and BQP is still an open question. We have already known that $BQP \subseteq PSPACE$ and $NP \subseteq PSPACE$.

Below is **my understanding of the relationship between NP and BQP** . Now, I need to import a vital concept called **QMA**.

• Definition of QMA:

A language L belongs to QMA if there exists a polynomial-time quantum verifier V and a polynomial $p(n)$ such

that:

- For all $x \in L$, there exists a quantum state $|\psi\rangle$ (called a witness state), such that the probability that V accepts the input $(|x\rangle, |\psi\rangle)$ is at least $\frac{2}{3}$.
- For all $x \notin L$, and for all quantum states $|\psi\rangle$ with at most $p(|x|)$ qubits, the probability that V accepts the input $(|x\rangle, |\psi\rangle)$ is at most $\frac{1}{3}$.

3.1 Relationship between QMA and NP in the Context of NP vs. BQP:

- **QMA as a Quantum Analogue of NP:**

If NP were contained in BQP, it would imply that quantum computers can solve all problems that classical computers can verify efficiently. This would be a significant result, as it would suggest that quantum computers have a broad advantage over classical computers.

However, the existence of QMA suggests that quantum verification (via QMA) is more powerful than classical verification (via NP). This implies that quantum computers may have unique capabilities in verification problems, even if they cannot solve all NP problems efficiently.

- **Open Questions and Conjectures:**

The fact that QMA is a quantum extension of NP raises the question of whether NP is contained in BQP. If NP were contained in BQP, it would imply that quantum computers can solve all NP problems efficiently, which is widely believed to be unlikely.

Many complexity theorists conjecture that $NP \nsubseteq BQP$, meaning there are problems in NP that cannot be solved efficiently by quantum computers. This conjecture is supported by the lack of efficient quantum algorithms for NP-complete problems, such as SAT (Boolean satisfiability).

- **Relativized Results:**

Relativized results in complexity theory suggest that there are oracles under which $NP \nsubseteq BQP$. These results provide evidence that NP and BQP are distinct classes in the real world, even though no unconditional proof exists yet.

The relationship between QMA and NP further supports this belief, as QMA is a quantum extension of NP and is believed to be strictly larger than NP.

3.2 Implications for NP vs. BQP:

- **Quantum Verification vs. Quantum Computation:**

QMA highlights that quantum computers can verify certain problems more efficiently than classical computers, but this does not necessarily imply that quantum computers can solve these problems more efficiently.

This distinction is crucial for understanding the relationship between NP and BQP. While quantum verification (QMA) may be more powerful than classical verification (NP), it does not imply that quantum computation (BQP) can solve all NP problems.

- **My Belief in $NP \nsubseteq BQP$:**

The belief that $NP \nsubseteq BQP$ is supported by the lack of efficient quantum algorithms for NP-complete problems and the existence of QMA, which suggests that quantum verification is more powerful than classical verification.

If $NP \nsubseteq BQP$, it would imply that quantum computers have limitations in solving certain problems that classical computers cannot solve efficiently.